LAPORAN PRAKTIKUM

POSTTEST 7

ALGORITMA PEMROGRAMAN DASAR



Disusun oleh:

Chika Pramulistia Putri 2609106006

Kelas A1 ‘26

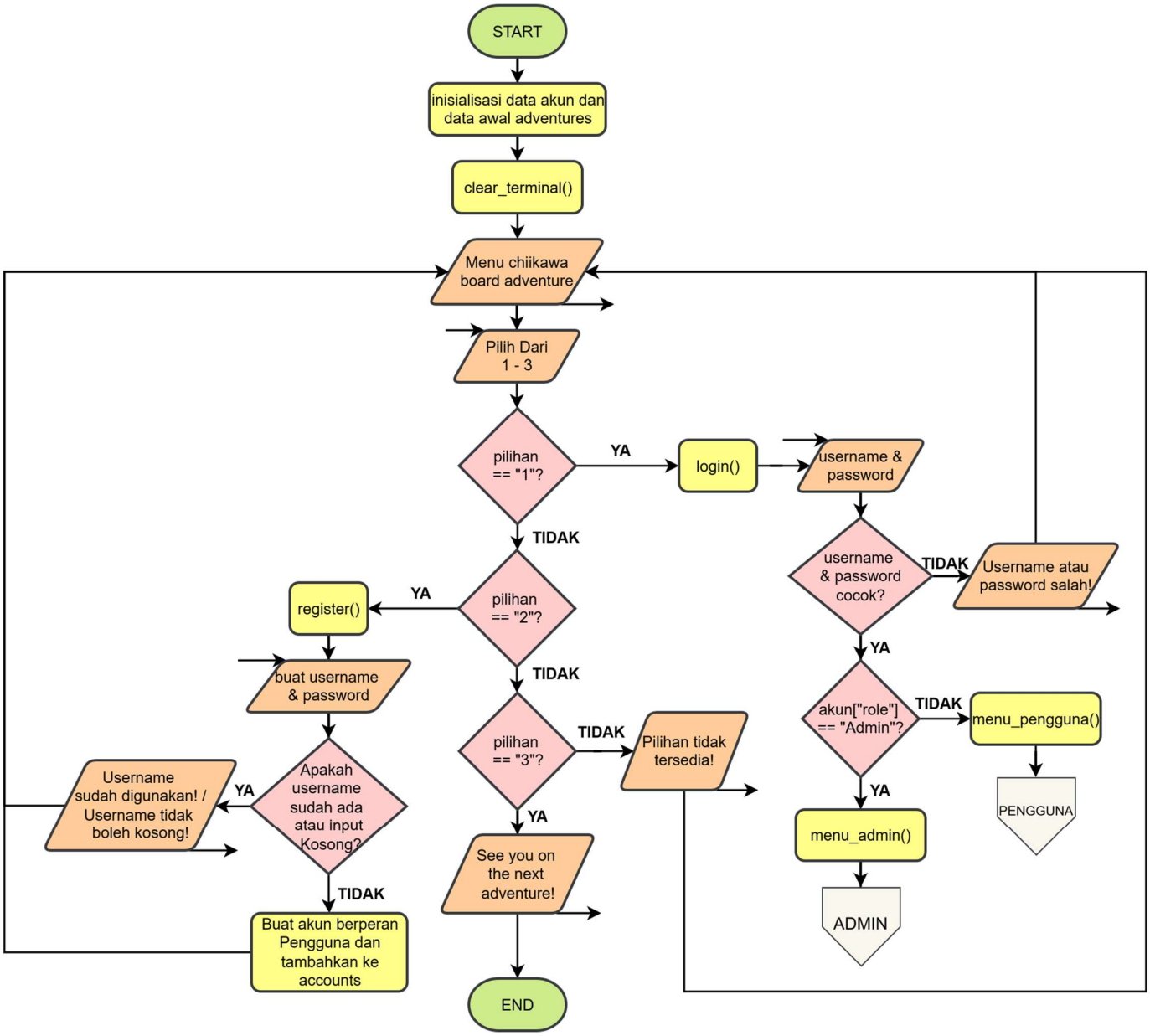
PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

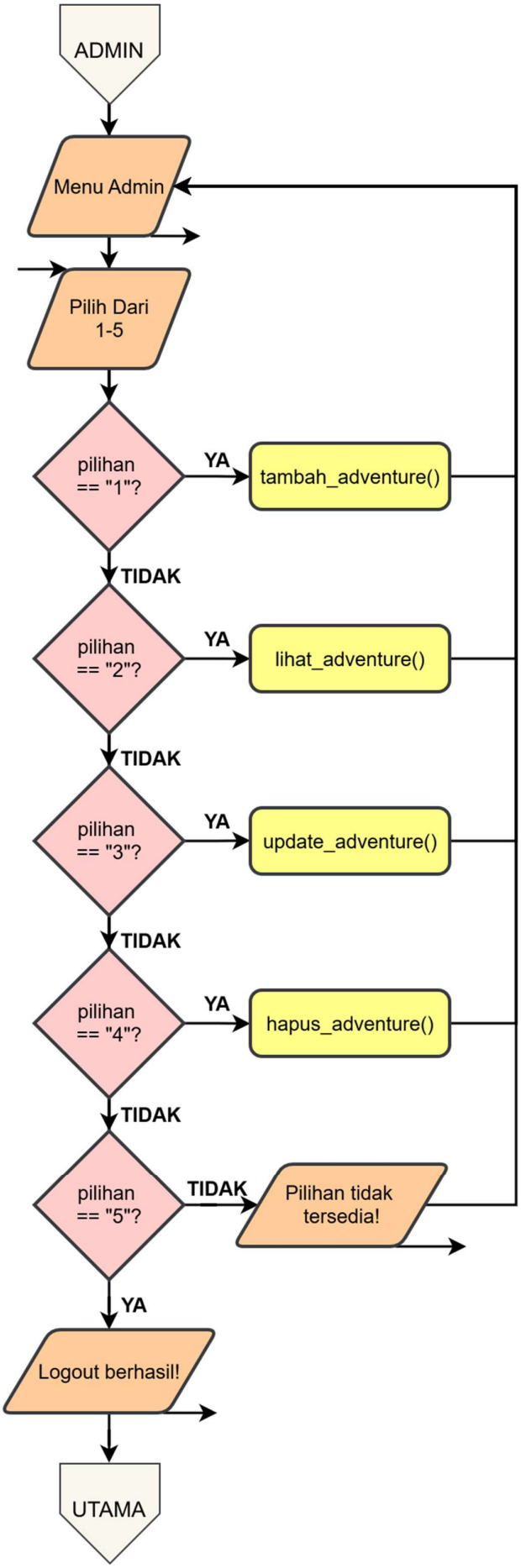
SAMARINDA

2026

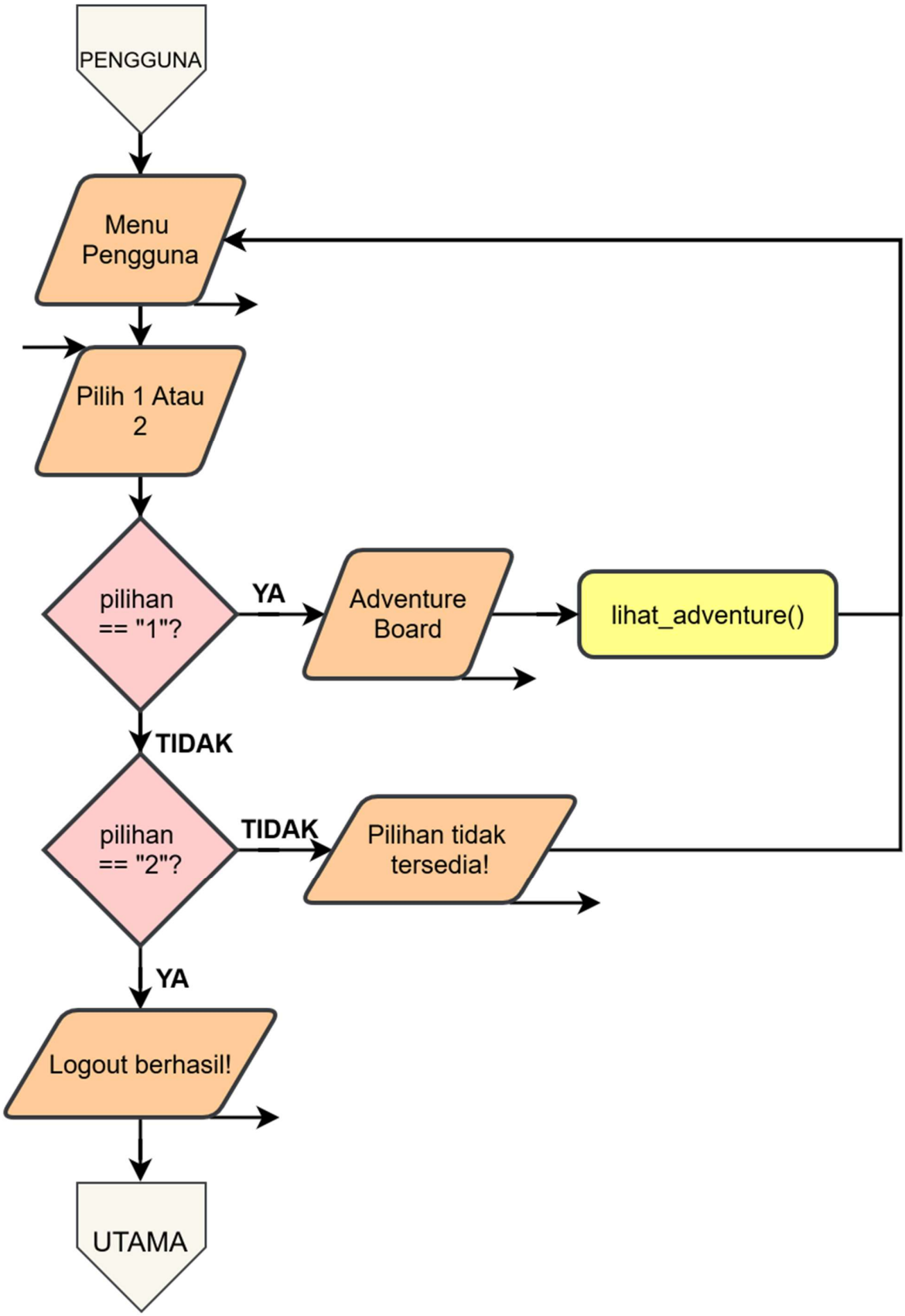
1. Flowchart



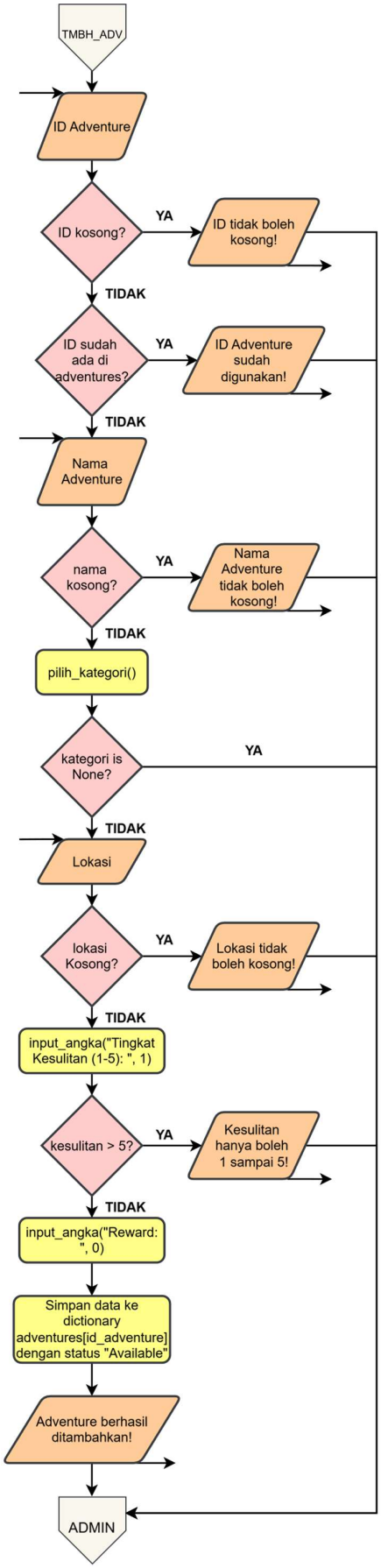
Gambar 1.1 Flowchart Menu Beranda Utama dan Otentikasi Akun Sistem



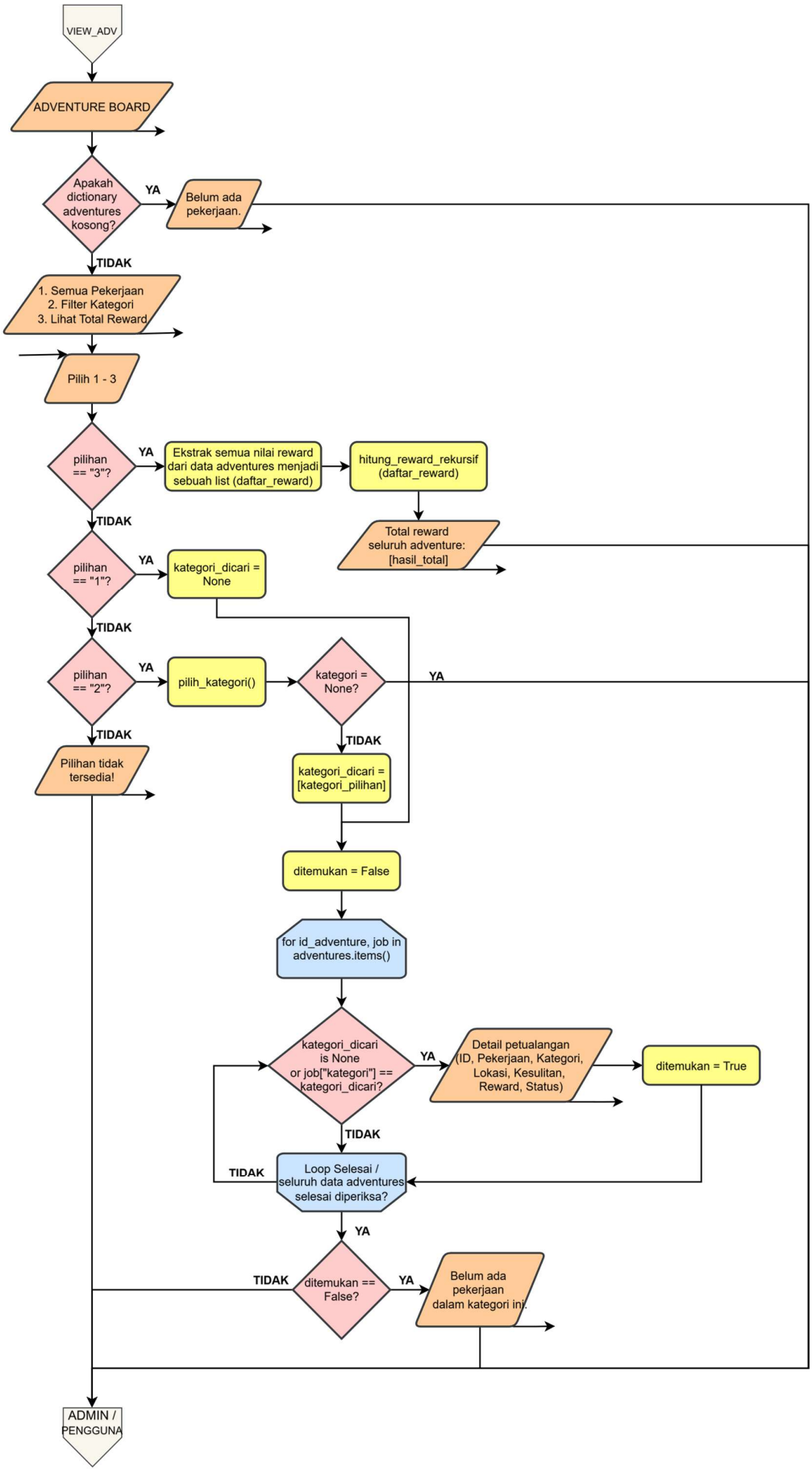
Gambar 1.2 Flowchart Navigasi Menu Interaktif Admin



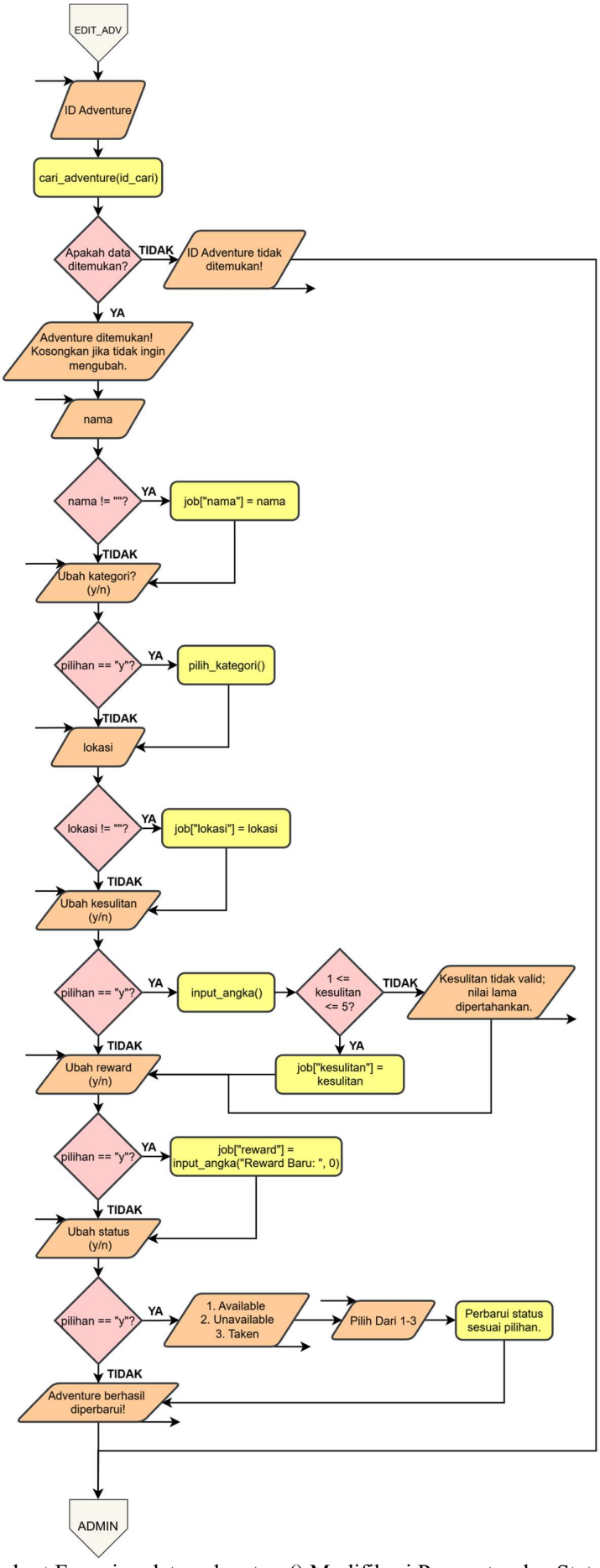
Gambar 1.3 Flowchart Navigasi Menu Interaktif Pengguna



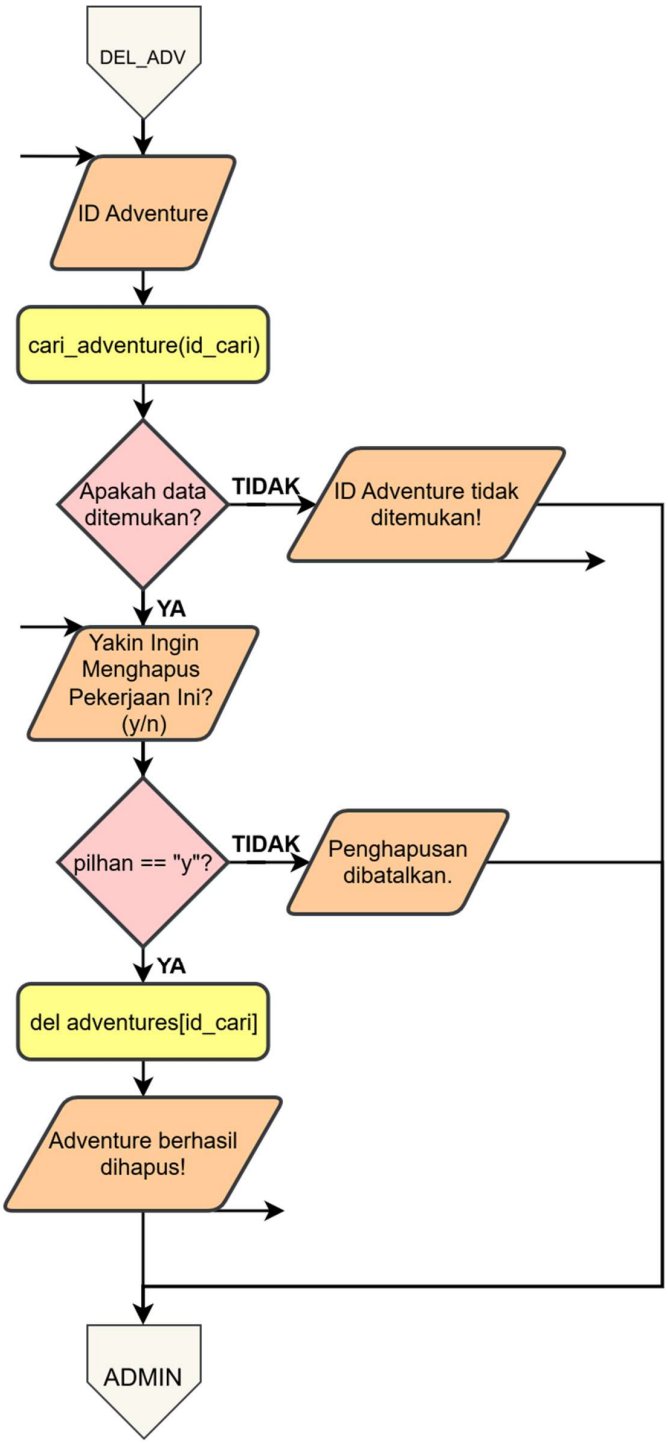
Gambar 1.4 Flowchart Fungsi tambah_adventure() Modul Operasi Penambahan Adventures Baru



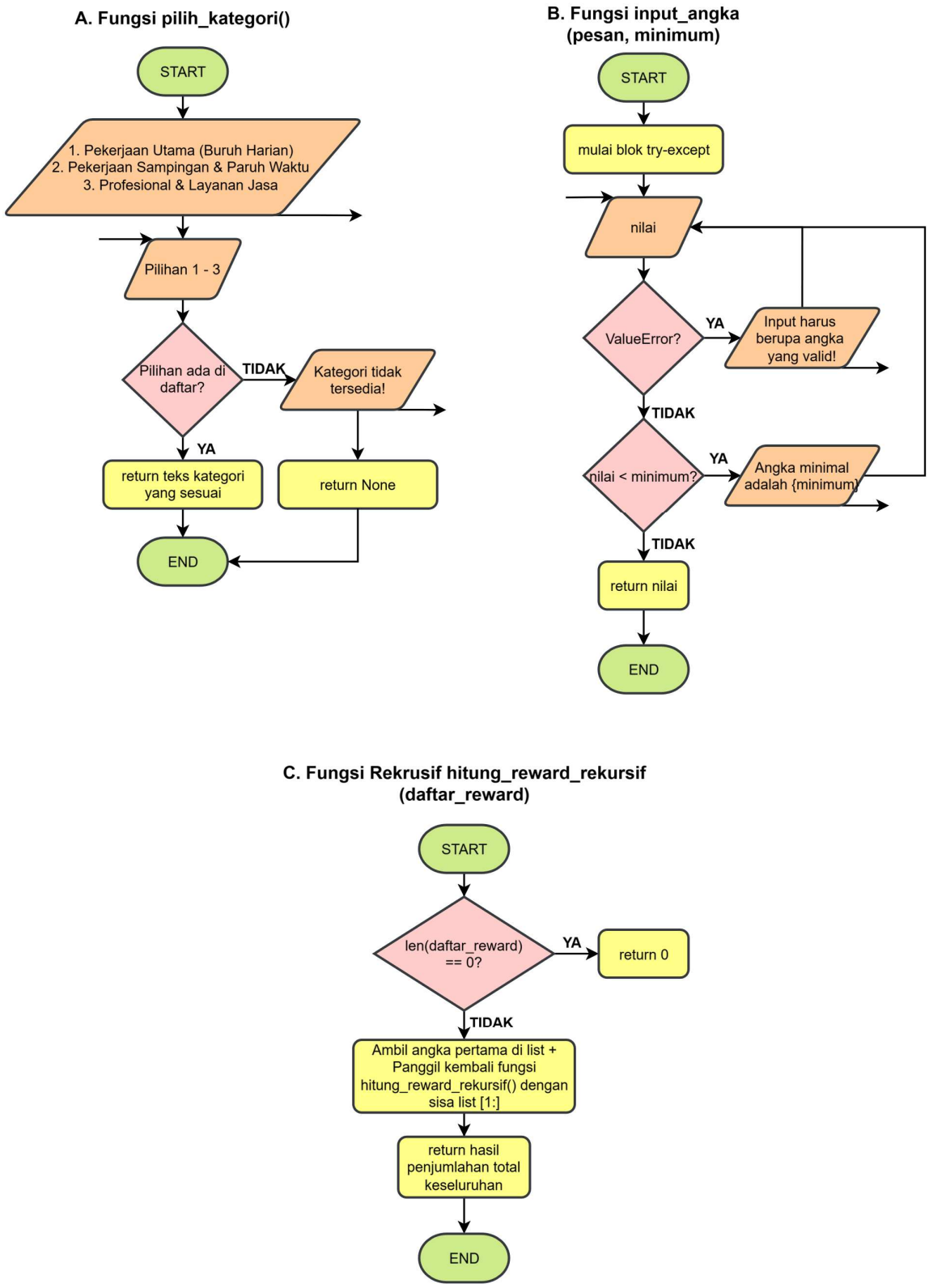
Gambar 1.5 Flowchart Fungsi lihat_adventure() Papan Mading Informasi dan Filter Kategori



Gambar 1.6 Flowchart Fungsi update_adventure() Modifikasi Parameter dan Status Misi Petualangan



Gambar 1.7 Flowchart Fungsi hapus_adventure() dengan Konfirmasi Keamanan Berlapis



Gambar 1.8 Flowchart Fungsi Pembantu (Helper Functions)

a) Penjelasan *Flowchart*

Flowchart tersebut menampilkan langkah-langkah operasional untuk aplikasi manajemen misi petualangan digital bertajuk "CHIIKAWA ADVENTURE BOARD". Pada pengerjaan Posttest 7 ini, program mengalami peningkatan arsitektur yang signifikan melalui *penerapan fungsi modular (def),* algoritma rekursif, serta sistem penanganan pengecualian ganda (*try-except*) untuk mencegah terjadinya kegagalan sistem (*crash*). Program ini dilengkapi dengan sistem validasi otentikasi akun, pembatasan parameter pengisian tingkat kesulitan dan reward, menu dasbor interaktif Admin maupun Pengguna, serta fitur penghitungan total bonus petualangan secara otomatis menggunakan fungsi rekursif.

Flowchart Beranda Utama dan Otentikasi

- Awal Alur (START): Diagram dimulai melalui simbol *terminator* START. Alur langsung mengarah pada proses inisialisasi database awal program, yaitu menyiapkan Dictionary accounts yang berisi data kredensial dan Dictionary adventures yang menampung daftar misi petualangan bawaan.
- Siklus Menu Utama: Program memanggil fungsi clear_terminal() untuk membersihkan layar terminal. Alur kemudian memasuki perulangan utama (*looping*) dengan menampilkan banner teks judul program beserta tiga menu pilihan: angka 1 untuk *Login*, angka 2 untuk *Register*, dan angka 3 untuk *Exit*. Proses input mengambil nilai dari terminal dan menyimpannya ke dalam variabel pilihan. Alur lalu masuk ke dalam rangkaian struktur kondisi penyeleksian (*Decision*):
    - Opsi Exit (Pilihan == "3"): Jika input bernilai "3" (YA), sistem menampilkan pesan perpisahan *"See you on the next adventure!"*, memutus rantai perulangan, dan bergerak lurus mengakhiri eksekusi program menuju END.
    - Opsi Register (Pilihan == "2"): Jika input bernilai "2" (YA), program memanggil fungsi register(). Alur masuk ke dalam blok registrasi di mana pengguna diminta menginput data ke terminal. Apabila string username atau password kosong, atau username terdeteksi sudah terdaftar di database accounts (*Decision* bernilai YA), sistem mengeluarkan notifikasi *error* dan mengembalikan alur ke menu utama. Jika lolos kualifikasi, data disimpan ke database dengan peran otomatis sebagai "Pengguna", memicu pesan *"Registrasi berhasil!"*, lalu alur kembali diarahkan ke menu utama.
    - Opsi Login (Pilihan == "1"): Jika input bernilai "1" (YA), program mengeksekusi fungsi login(). Pengguna memasukkan data string username dan password. Tahap *Decision* memeriksa kecocokan data dengan database accounts. Jika data tidak valid (TIDAK), notifikasi *"Username atau password salah!"* akan dicetak dan alur berputar

kembali ke menu utama. Jika valid (YA), login dinyatakan sukses dan sistem melakukan penyeleksian hak akses berdasarkan peran akun (role). Apabila akun terdeteksi sebagai "Admin", alur melompat keluar halaman melalui *Off-Page Connector* "ADMIN". Sedangkan jika peran akun adalah "Pengguna", alur dialihkan keluar halaman melalui *Off-Page Connector* "PENGGUNA".

- Input Tidak Valid**:** Jika input pilihan menu beranda utama di luar angka 1, 2, atau 3, sistem otomatis menampilkan notifikasi *"Pilihan tidak tersedia!"* dan memutar kembali panah alurnya ke atas menuju menu utama.

Flowchart Dasbor Informasi Pengguna

Ketika proses otentikasi login berhasil mendeteksi peran akun petualang sebagai "Pengguna", sistem akan mengalihkan alur keluar dari halaman depan melalui sambungan *Off-Page Connector* "Pengguna" dan langsung mengeksekusi fungsi menu_pengguna(). Di dalam modul ini, langkah pertama yang dilakukan adalah menjalankan perintah pembersihan halaman terminal secara otomatis sebelum menyajikan halaman dasbor petualang interaktif yang menampung masukan pengguna melalui variabel pilihan, di mana angka 1 ditujukan untuk Lihat Board dan angka 2 untuk *Logout*. Apabila pengguna memasukkan opsi angka "2" (YA pada validasi *logout*), sistem akan langsung mencetak Output teks "Logout berhasil!" ke terminal dan menarik garis alur keluar dari halaman pengguna memanfaatkan *Off-Page Connector* "UTAMA" demi memutar kembali navigasi ke beranda paling depan aplikasi.

Namun, jika pengguna memasukkan opsi angka "1" (YA pada penyeleksian lihat board), program secara linear akan mengaktifkan fungsi lihat_adventure() yang memicu Output tampilan sub-menu papan petualangan baru berisi tiga instruksi kontrol, yaitu angka 1 untuk Semua Pekerjaan, angka 2 untuk Filter Berdasarkan Kategori, dan angka 3 untuk Lihat Total Reward. Jika pengguna menjatuhkan pilihan pada opsi "3", program secara linear langsung mengeksekusi fungsi tampilkan_total_reward() untuk mengekstrak seluruh nilai bonus dari database menjadi sebuah variabel daftar_reward berbentuk list array, yang kemudian dilemparkan secara terpusat menuju fungsi rekursif hitung_reward_rekursif(). Di dalam siklus fungsi rekursif tersebut, sistem akan terus memecah array dan menjumlahkan setiap elemen numerik secara berulang hingga kondisi batas list kosong terpenuhi, untuk kemudian memunculkan Output hasil kalkulasi akumulasi total bonus petualangan secara presisi di layar.

Di sisi lain, jika pengguna memasukkan opsi angka "1" pada mading petualangan, program akan mengatur variabel kategori_dicari = None, sedangkan jika pengguna memilih opsi angka "2", sistem akan memicu fungsi pembantu pilih_kategori() untuk menetapkan nilai string kategori target berdasarkan angka paket yang dipilih petualang (Pekerjaan Utama, Sampingan, atau Profesional).

Setelah salah satu dari kondisi penyeleksian awal mading tersebut terpenuhi, alur kerja program akan bergerak menetapkan bendera penanda ditemukan = False dan langsung memasuki struktur perulangan *for loop* global for id_adventure, job in adventures.items(). Di dalam tahapan *looping* ini, sistem secara adaptif akan menyisir setiap baris data di memori dan menerapkan penyeleksian kondisi murni; apabila variabel kategori bernilai *None* atau parameter kategori pada data misi terbukti cocok dengan data yang dicari, sistem secara otomatis langsung mengeluarkan Output bukti cetak spesifikasi detail petualangan (ID, Pekerjaan, Kategori, Lokasi, Kesulitan, Reward, dan Status) secara rapi sekaligus mengubah status bendera penanda menjadi *True*.

Setelah perulangan global selesai memeriksa seluruh isi dictionary secara tuntas, alur bergerak menuju gerbang *Decision* akhir untuk mengecek kondisi bendera penanda; jika statusnya tetap bernilai *False*, sistem akan memunculkan Output peringatan "Belum ada pekerjaan dalam kategori ini.", namun jika bernilai *True*, alur akan dilewatkan secara aman. Akhir dari seluruh rangkaian proses pencetakan mading informasi tersebut—baik melalui skema penayangan global, penyaringan filter kategori berjenjang, maupun operasi hitung bonus total rekursif—akan ditutup dengan menarik seluruh ujung panah diagram berputar kembali ke atas menuju halaman awal dasbor menu pengguna secara interaktif.

Flowchart Dasbor Operasional Admin

Ketika alur masuk melalui sambungan *Off-Page Connector* "ADMIN", sistem langsung mengeksekusi pembersihan layar terminal dan mengaktifkan perulangan utama dasbor Admin via fungsi menu_admin(). Seluruh blok fungsi ini dilindungi oleh sistem pengaman ganda try-except untuk menangkap galat spesifik berupa KeyError dan TypeError, sehingga apabila terjadi kesalahan indeks data, sistem secara adaptif akan menangkapnya, mencetak teks peringatan galat ke layar, dan menjaga agar aplikasi tidak keluar secara paksa. Dasbor kontrol ini menyajikan menu interaktif CRUD penuh dengan lima pilihan angka, yaitu angka 1 untuk *Add Adventure*, angka 2 untuk *View Adventure*, angka 3 untuk *Edit Adventure*, angka 4 untuk *Delete Adventure*, dan angka 5 untuk *Exit*. Proses Input akan mengambil nilai string dari terminal dan menyimpannya ke dalam variabel pilihan untuk dialirkan menuju rangkaian gerbang penyeleksian kondisi (*Decision*). Jika Admin memasukkan opsi angka "5" (YA pada validasi keluar), sistem otomatis memutus rantai perulangan dasbor, memicu Output teks "Logout berhasil!", dan melempar alur kembali menuju halaman depan beranda utama melalui sambungan *Off-Page Connector* "UTAMA".

Namun, jika Admin menjatuhkan pilihan pada opsi angka "1" (YA), alur dialihkan secara linear untuk memanggil fungsi tambah_adventure() guna membuka halaman pembuatan formulir pekerjaan baru. Proses diawali dengan meminta Input kode string ID Adventure yang langsung divalidasi oleh gerbang *Decision*; apabila string kosong atau kodenya terdeteksi sudah digunakan di dalam database adventures, sistem langsung menolak data, memunculkan notifikasi *error*, dan memutar alur kembali ke dasbor utama Admin. Jika kode ID dinyatakan aman, Admin dituntun mengisi formulir data teks berupa nama petualangan, sub-kategori melalui fungsi pilih_kategori(), serta lokasi misi. Tahap krusial berikutnya adalah pengisian tingkat kesulitan dan jumlah hadiah (*reward*) numerik yang dialihkan secara modular memanfaatkan fungsi validasi terpusat input_angka(). Di dalam fungsi pembantu tersebut, alur Admin akan dikunci oleh perulangan lokal dan perangkap eksepsi try-except ValueError sehingga apabila Admin memasukkan karakter non-digit berupa huruf atau batas rentang nilainya tidak valid, sistem langsung menolak input di tempat dan memaksa Admin mengisi ulang hingga benar. Setelah seluruh parameter lolos kualifikasi, data terekam secara permanen ke dictionary adventures dengan default status "Available", memunculkan Output sukses ditambahkan, lalu panah alurnya memutar kembali ke dasbor utama Admin.

Sementara itu, jika Admin memasukkan opsi angka "2" (YA), program secara linear akan memanggil fungsi lihat_adventure() yang memiliki alur logika pemrosesan data, penyaringan filter kategori berjenjang, dan perhitungan akumulasi bonus total memanfaatkan algoritma fungsi rekursif hitung_reward_rekursif() yang berjalan sama persis seperti modul mading pada halaman pengguna, diakhiri dengan pemutaran panah kembali ke dasbor Admin. Selanjutnya, jika Admin memilih opsi angka "3" (YA), sistem akan memicu fungsi update_adventure() dan meminta masukan string kode ID Adventure target yang

posisinya akan dicari di memori lewat fungsi pembantu cari_adventure(). Apabila kode ID tidak ditemukan pada database *nested dictionary*, sistem mencetak Output error dan mengembalikan alur ke dasbor utama, namun jika *Decision* kecocokan bernilai YA, Admin langsung dihadapkan pada modul sunting data parsial (Nama Baru, Kategori Baru, Lokasi Baru, Kesulitan Baru, Reward Baru, dan Status Baru). Modul penyuntingan ini dirancang dengan tingkat fleksibilitas tinggi memanfaatkan seleksi kondisi string kosong, di mana Admin cukup langsung menekan tombol Enter tanpa mengisi teks apa pun jika tidak berniat mengubah nilai lama data tersebut. Sesi pembaruan status pekerjaan juga disuguhkan secara interaktif menggunakan menu pilihan sub-angka (1. Available, 2. Unavailable, 3. Taken), dan setelah seluruh rangkaian parameter selesai dimodifikasi, sistem memicu Output "Adventure berhasil diperbarui!" dan menarik ujung panah alur kembali ke halaman dasbor Admin.

Terakhir, jika Admin memasukkan opsi angka "4" (YA), program akan mengeksekusi fungsi hapus_adventure() dengan meminta input data target ID Adventure yang akan dicari keberadaannya di database. Jika ID tidak ditemukan, sistem mengeluarkan peringatan dan memutar kembali alur, tetapi jika data ditemukan, alur diarahkan masuk ke gerbang keamanan konfirmasi akhir berupa pilihan opsi string (y/n) lewat variabel konfirmasi. Jika Admin memasukkan karakter "y" (YA pada gerbang konfirmasi), proses penghapusan data internal dengan perintah del adventures[id_cari] langsung dieksekusi secara permanen untuk mengosongkan memory, diikuti dengan Output teks "Adventure berhasil dihapus!", sementara jika memilih opsi "n" atau karakter lain (TIDAK), sistem akan mencetak teks "Penghapusan dibatalkan.". Akhir dari seluruh percabangan operasional hapus berkas mading tersebut akan ditutup secara seragam dengan menarik garis panah diagram berputar kembali ke atas menuju dasbor utama menu Admin.

Flowchart Fungsi Pembantu (Helper Functions)

- **Fungsi pilih_kategori()**

  Prosedur fungsi pembantu pilih_kategori() diawali melalui simbol *terminator* **START** dan langsung bergerak linear mengeksekusi Output pencetakan mading paket pilihan kategori di terminal, yaitu angka 1 untuk Pekerjaan Utama (Buruh Harian), angka 2 untuk Pekerjaan Sampingan & Paruh Waktu, serta angka 3 untuk Profesional & Layanan Jasa. Proses selanjutnya adalah mengambil data masukan string dari terminal via variabel pilihan yang telah dibersihkan menggunakan metode .strip(). Alur kemudian masuk ke dalam gerbang *Decision* penyeleksian kondisi untuk memeriksa apakah string pilihan terdaftar di dalam dictionary lokal fungsi yang bertindak sebagai database pemetaan kategori. Jika hasil pemeriksaan *Decision* bernilai YA, sistem secara linear akan mengeksekusi perintah return untuk mengembalikan teks string nama kategori utuh yang sesuai menuju fungsi utama yang memanggilnya, lalu alur diarahkan lurus mengakhiri proses fungsi pada *terminator* **END**. Sebaliknya, jika data input yang dimasukkan Admin tidak terdaftar di dalam dictionary lokal (Decision bernilai TIDAK), alur otomatis dialihkan menuju Output peringatan teks "Kategori tidak tersedia!", dilanjutkan dengan perintah return None, dan ditarik lurus mengakhiri jalannya diagram menuju *terminator* **END**.

- **Fungsi input_angka(pesan, minimum)**

  Diagram fungsi pembantu input_angka() dimulai pada simbol *terminator* **START** dan dirancang bekerja di dalam siklus perulangan tak terbatas while True. Alur langsung diarahkan masuk menuju blok pengaman pengolahan data *try-except* untuk mengantisipasi kegagalan sistem akibat salah tipe data. Di dalam blok *try*, proses Input mengambil data dari terminal melalui variabel nilai yang dipaksa langsung dikonversi menjadi tipe data integer (int()). Apabila pengguna memasukkan karakter non-digit berupa huruf atau simbol, sistem secara otomatis akan memicu eksepsi *ValueError*, sehingga alur melompat ke blok *except* untuk memunculkan Output peringatan teks "Input harus berupa angka yang valid!" dan menarik kembali panah alur memutar ke atas perulangan. Namun, jika data lolos dari perangkap tipe data, alur bergerak menuju gerbang *Decision* penyeleksian batas; apabila variabel nilai terdeteksi lebih kecil dari parameter minimum (Decision bernilai YA), sistem mengeluarkan Output notifikasi "Angka minimal adalah {minimum}!" dan alur diputar kembali ke atas. Jika seluruh gerbang kualifikasi terpenuhi di mana nilai telah valid dan berada di atas batas minimal (Decision bernilai TIDAK), sistem mengeksekusi perintah return nilai untuk mengembalikan data numerik murni tersebut keluar dari fungsi, memutus rantai perulangan secara otomatis, dan mengakhiri seluruh jalannya diagram fungsi menuju *terminator* **END**.

- **Fungsi Rekursif hitung_reward_rekursif(daftar_reward)**

  Diagram alur fungsi rekursif hitung_reward_rekursif() diawali dari simbol *terminator* **START** dengan menerima sebuah parameter masukan berupa list array yang menampung data numerik reward. Alur langsung dihadapkan pada gerbang *Decision* krusial yang bertindak sebagai kondisi batas (*base case*) rekursif, yaitu mengecek apakah panjang dari list array daftar_reward sudah bernilai sama dengan 0 (len(daftar_reward) == 0). Jika kondisi batas tersebut terpenuhi (Decision bernilai YA), alur akan bergerak linear mengeksekusi perintah return 0 untuk menghentikan siklus pemanggilan mandiri dan langsung ditarik lurus menuju *terminator* **END**. Namun, apabila panjang list array terdeteksi masih berisi elemen numerik atau belum sama dengan nol (Decision bernilai TIDAK), fungsi akan mengaktifkan kondisi rekursif (*recurrence step*) dengan mengeksekusi proses pembongkaran array. Proses ini dilakukan dengan mengambil nilai angka pada indeks pertama list (daftar_reward[0]) untuk dijumlahkan secara akumulatif dengan memanggil kembali fungsi hitung_reward_rekursif() secara mandiri, namun dengan melemparkan sisa elemen list yang telah dipotong mulai dari indeks kedua hingga akhir (daftar_reward[1:]). Siklus tumpukan pemanggilan mandiri ini akan terus berputar dan mengunci alur fungsi di memori hingga seluruh elemen list habis habis terkalkulasi, untuk kemudian mengeksekusi perintah return hasil penjumlahan total keseluruhan data finansial secara presisi dan bergerak lurus mengakhiri diagram menuju *terminator* **END**.

2. Deskripsi Singkat Program

Program ini merupakan sistem aplikasi papan informasi petualangan digital bertajuk **"CHIIKAWA ADVENTURE BOARD"** yang dirancang untuk memproses manajemen data penawaran pekerjaan (misi petualangan) berbasis multi-user (*Admin* dan *Pengguna*). Pada pengerjaan Posttest 7 ini, arsitektur kode program dikembangkan secara menyeluruh dari yang sebelumnya berupa baris perulangan linier biasa menjadi **berbasis fungsi modular terstruktur (def)**. Selain itu, program ini mengintegrasikan **algoritma fungsi rekursif** untuk kalkulasi data finansial serta **sistem penanganan pengecualian ganda (*Advanced Error Handling* menggunakan blok try-except)**. Hal ini bertujuan untuk mengamankan jalannya eksekusi program di dalam terminal dari potensi kegagalan sistem (*crash*) akibat kesalahan tipe data input (*ValueError*), ketidaksesuaian indeks *array* (*KeyError*), ketidakcocokan tipe parameter (*TypeError*), hingga aksi penutupan paksa aplikasi secara mendadak oleh pengguna (*KeyboardInterrupt* dan *EOFError*).

a) Alur Kerja Program

Alur kerja program diawali dengan eksekusi fungsi pembersih halaman terminal dan langsung menampilkan menu beranda awal sistem yang meminta pengguna memilih opsi masuk aplikasi. Melalui menu registrasi, sistem akan memvalidasi agar string data username baru tidak boleh kosong serta wajib bersifat unik atau belum terdaftar di database dictionary accounts. Pada menu masuk, identitas diperiksa silang secara langsung. Jika data otentikasi valid, program membaca hak peran akses petualang. Apabila masuk sebagai peran "Admin", sistem akan membuka modul menu kontrol khusus yang mengizinkan operasi CRUD penuh melalui fungsi modular terstruktur, meliputi membuat misi baru, melihat papan mading petualangan, menyunting detail spesifikasi pekerjaan, dan menghapus penawaran misi secara permanen. Sebaliknya, jika login sebagai peran "Pengguna", sistem hanya akan membuka akses modul mading baca-saja (*read-only*) untuk mengamati ketersediaan pekerjaan di papan petualangan.

Dalam pengerjaan operasional data misi petualangan, sistem menerapkan standarisasi tipe data dan batasan nilai yang ketat melalui fungsi modular terpusat. Pengisian tingkat kesulitan misi dibatasi wajib berada di dalam rentang angka integer 1 sampai 5, sedangkan nilai total reward bonus hadiah diatur minimal bernilai angka 0. Setiap kali terjadi kesalahan pengisian pada parameter numerik tersebut, baik karena bernilai di luar batas rentang maupun akibat masukan karakter non-digit berupa huruf, sistem perulangan while True lokal yang dipadukan dengan

perangkap eksepsi try-except ValueError secara adaptif akan menolak data, memunculkan notifikasi teks error, dan langsung mengunci pengguna untuk mengisi ulang di tempat hingga valid tanpa harus mengulang formulir dari awal. Papan informasi pekerjaan juga menyediakan opsi klasifikasi filter agar pengguna dapat menyaring tampilan berdasarkan tiga kategori utama pekerjaan (Pekerjaan Utama, Pekerjaan Sampingan, atau Profesional) secara rapi. Selain itu, program menyediakan fitur kalkulasi finansial otomatis di mana saat menu total reward dipilih, sistem akan memanggil fungsi rekursif untuk mengakumulasikan seluruh nilai bonus petualangan secara presisi. Sesi navigasi aplikasi akan terus berputar secara interaktif di dalam terminal menggunakan kendali perulangan tak terbatas while True, hingga pengguna secara sadar memilih menu keluar (Exit) yang akan memicu pencetakan pesan penutup aplikasi dan memutus rantai proses eksekusi kode program secara aman dan bersih.

3. Source Code

A. Fitur Kontrol Validasi Input Angka dan Error Handling Modular Berbasis try-except

Fitur ini dibungkus ke dalam fungsi modular input_angka() untuk memastikan bahwa setiap input numerik (seperti tingkat kesulitan dan reward) wajib berupa angka murni yang valid dan mematuhi batas aturan minimum. Fitur ini memanfaatkan blok try-except ValueError untuk menangkap input non-digit agar program tidak mengalami *crash*.

Source Code:

```
def input_angka(pesan, minimum):
    while True:
        try:
            nilai = int(input(pesan))

            if nilai < minimum:
                print(f"Angka minimal adalah {minimum}!")
                continue

            return nilai

        except ValueError:
            print("Input harus berupa angka yang valid!")
```

B. Fitur Kalkulasi Finansial Otomatis Menggunakan Algoritma Fungsi Rekursif

Fitur ini diterapkan pada fungsi hitung_reward_rekursif() untuk mengakumulasikan total hadiah (*reward*) dari seluruh petualangan. Fungsi ini bekerja dengan memanggil dirinya sendiri (*self-invocation*) dan memotong list secara berulang ([1:]) hingga mencapai *base case* (panjang list sama dengan nol).

Source Code:

```
def hitung_reward_rekursif(daftar_reward):
    if len(daftar_reward) == 0:
       return 0
    return daftar_reward[0] + hitung_reward_rekursif(daftar_reward[1:])

def tampilkan_total_reward():
    daftar_reward = [job["reward"] for job in adventures.values()]
    total_reward = hitung_reward_rekursif(daftar_reward)
    print(f"Total reward seluruh adventure: {total_reward}")
```

C. Fitur Proteksi Duplikasi Identitas Akun Baru pada Dictionary

Fitur modular ini disematkan di dalam fungsi register() untuk mengunci string masukan agar tidak terjadi penumpukan atau duplikasi nama pengguna (*username*) terdaftar di dalam basis data *dictionary* accounts.

Source Code:

```
username = input("Buat Username: ")
if not username:
    print("Username tidak boleh kosong!")
    return
if username in accounts:
    print("Username sudah digunakan!")
    return
password = input("Buat Password: ")
if not password:
    print("Password tidak boleh kosong!")
    return
accounts[username] = {"password": password, "role": "Pengguna"}
print("Registrasi berhasil!")
```

D. Fitur Penyaringan Informasi Berjenjang via Ekstrak Data Mading (.items)

Fitur ini berfungsi memilah susunan isi papan informasi mading petualangan menggunakan perulangan metode .items() secara kondisional berdasarkan kecocokan parameter string kategori khusus di dalam *nested dictionary*.

Source Code:

```
for id_adventure, job in adventures.items():
    if kategori_dicari is None or job["kategori"] == kategori_dicari:

       print("-----------------------------------")
       print(f"ID         : {id_adventure}")
       print(f"Pekerjaan  : {job['nama']}")
       print(f"Kategori   : {job['kategori']}")
       print(f"Lokasi     : {job['lokasi']}")
       print(f"Kesulitan  : {job['kesulitan']}")
       print(f"Reward     : {job['reward']}")
       print(f"Status     : {job['status']}")
```

E. Fitur Sunting Parsial Nilai Array Database Terpilih

Fitur ini digunakan untuk memodifikasi parameter nilai di dalam *dictionary* secara dinamis, di mana sistem memberikan fleksibilitas agar user cukup menekan tombol Enter (mengosongkan string "") jika tidak berniat mengubah nilai lama data tersebut.

Source Code:

```
    nama = input("Nama Baru: ")
    if nama != "":
        job["nama"] = nama

    print("Ubah kategori? (y/n)")
```

```
    if input("Pilihan: ") == "y":
        kategori = pilih_kategori()
        if kategori is not None:
            job["kategori"] = kategori
```

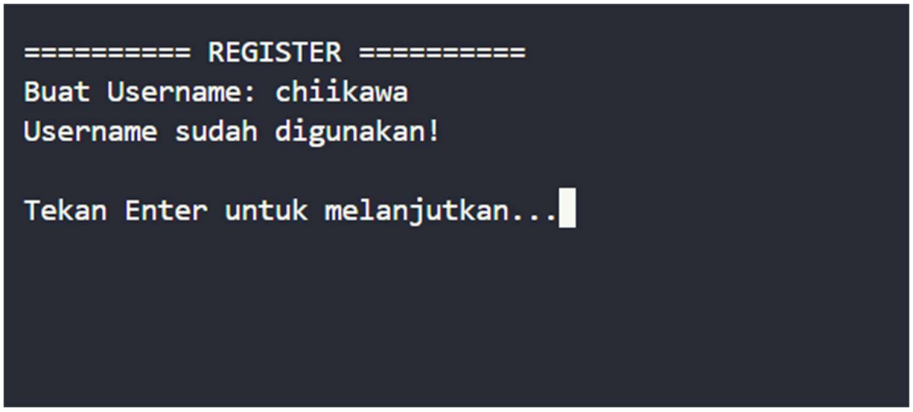
F. Fitur Pengaman Operasi Hapus Data Berbasis Kata Kunci *del*

Fitur ini bertindak sebagai gerbang konfirmasi pengaman sebelum sistem mengeksekusi penghapusan elemen data permanen dari basis data *dictionary* adventures menggunakan keyword internal python *del*.

Source Code:

```
    print("Yakin Ingin Menghapus Pekerjaan Ini? (y/n): ")
    if input("Pilihan: ") == "y":
        del adventures[id_cari]
        print("Adventure berhasil dihapus!")
    else:
        print("Penghapusan dibatalkan.")
```

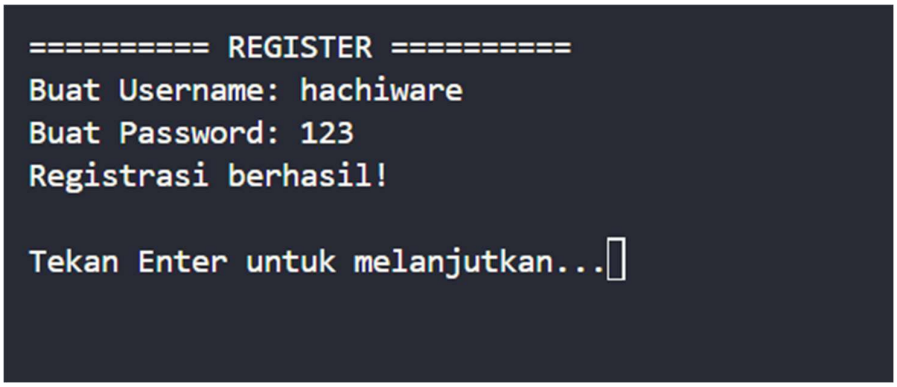
4. Hasil Output



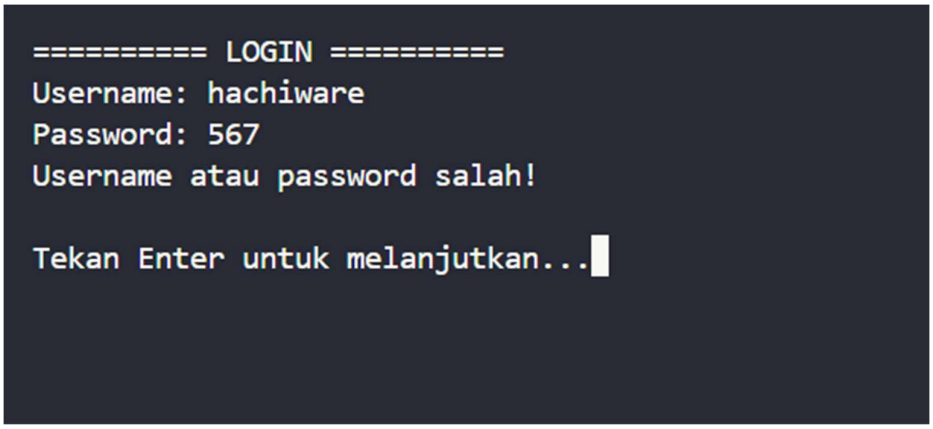
Gambar 4.1 Output Tampilan Menu Utama Beranda *Chiikawa Adventure Board* dan Uji Coba Pilihan 3



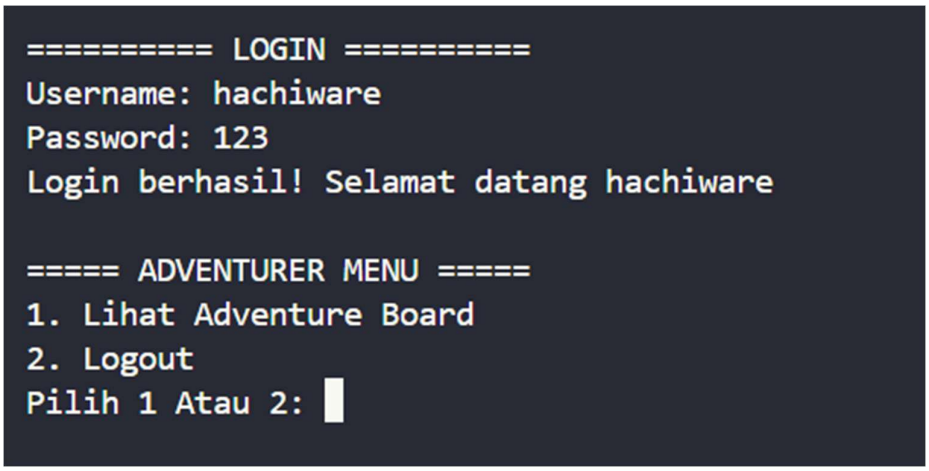
Gambar 4.2 Output Peringatan Sistem Jika *Username* Terdeteksi Duplikat



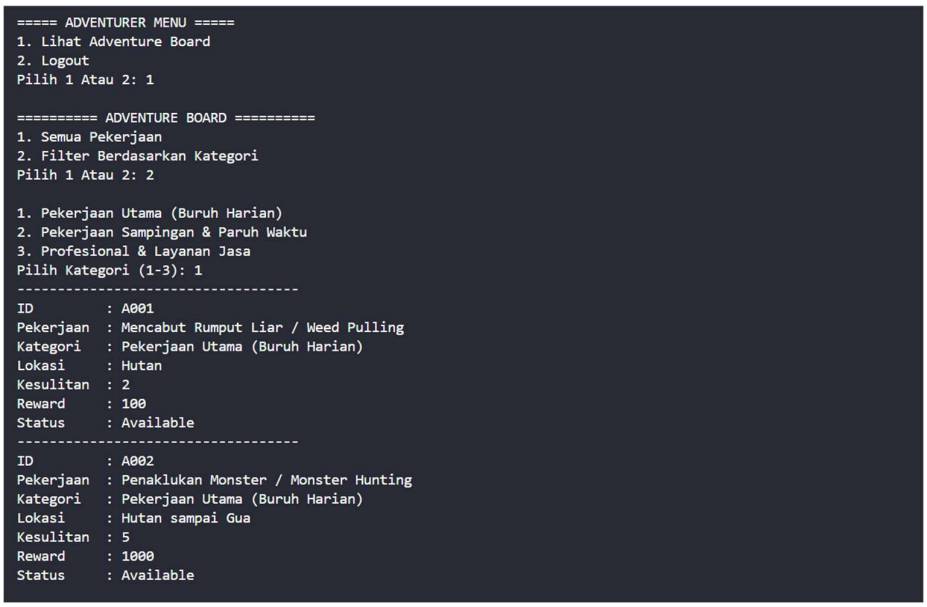
Gambar 4.3 Output Keberhasilan Registrasi Akun Petualang Baru



Gambar 4.4 Output Peringatan Sistem Pada *Login* Jika *Username* atau *Password* Salah



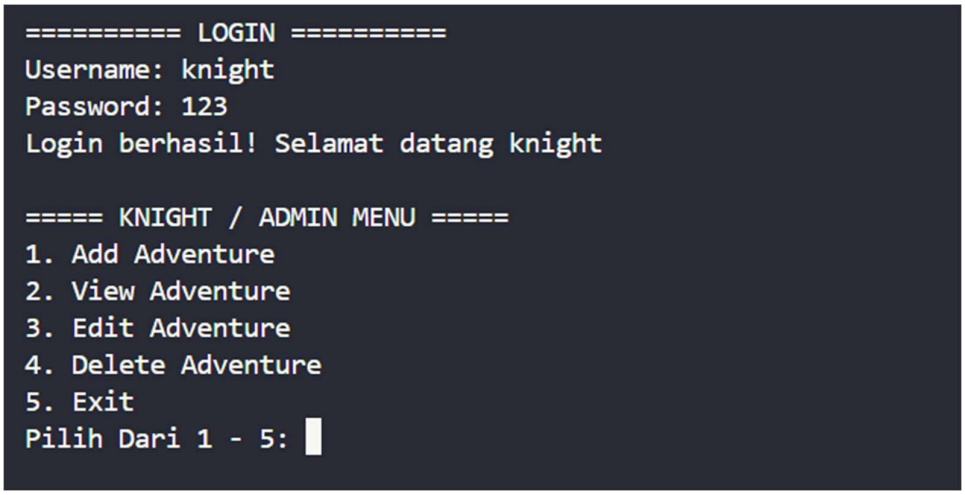
Gambar 4.5 Output Keberhasilan Login dan Tampilan Dasbor Untuk Peran Pengguna



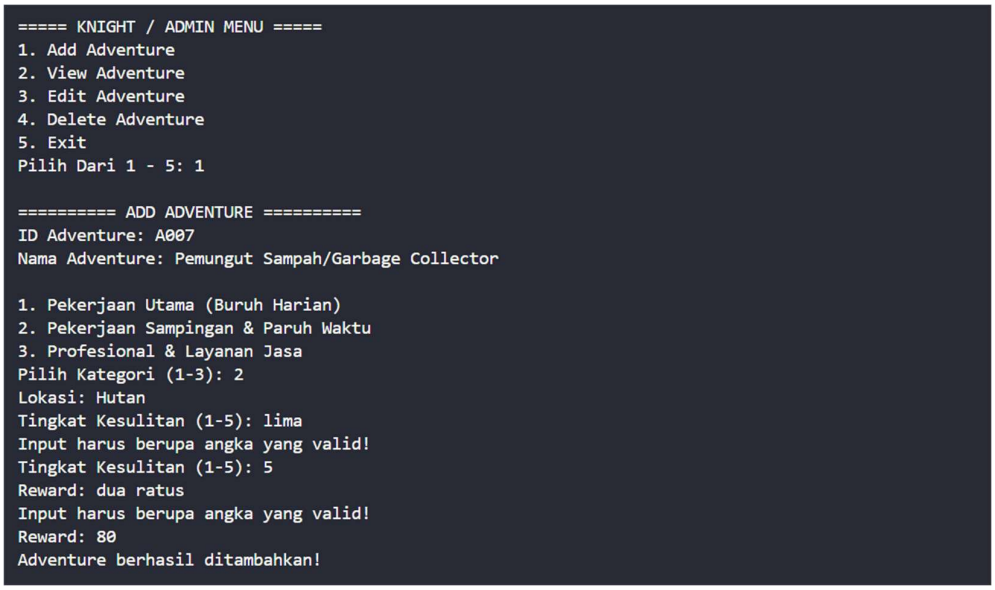
Gambar 4.6 Output Hasil Penyaringan (*Filtering*) Tampilan Papan Mading Berdasarkan Kategori Spesifik

```
========== ADVENTURE BOARD ==========
1. Semua Pekerjaan
2. Filter Berdasarkan Kategori
Pilih 1 Atau 2: 1
-----------------------------------
ID         : A001
Pekerjaan  : Mencabut Rumput Liar / Weed Pulling
Kategori   : Pekerjaan Utama (Buruh Harian)
Lokasi     : Hutan
Kesulitan  : 2
Reward     : 100
Status     : Available
-----------------------------------
ID         : A002
Pekerjaan  : Penaklukan Monster / Monster Hunting
Kategori   : Pekerjaan Utama (Buruh Harian)
Lokasi     : Hutan sampai Gua
Kesulitan  : 5
Reward     : 1000
Status     : Available
-----------------------------------
ID         : A003
Pekerjaan  : Menempel Stiker / Stickering Lemons
Kategori   : Pekerjaan Sampingan & Paruh Waktu
Lokasi     : Pabrik
Kesulitan  : 1
Reward     : 80
Status     : Available
-----------------------------------
ID         : A004
Pekerjaan  : Memetik Jamur / Mushroom Picking
Kategori   : Pekerjaan Sampingan & Paruh Waktu
Lokasi     : Hutan
Kesulitan  : 3
Reward     : 100
Status     : Available
-----------------------------------
ID         : A005
Pekerjaan  : Pegawai Kedai Ramen
Kategori   : Pekerjaan Profesional & Layanan Jasa
Lokasi     : Kedai Ramen
Kesulitan  : 4
Reward     : 800
Status     : Available
-----------------------------------
ID         : A006
Pekerjaan  : Pengatur Lalu Lintas / Polisi
Kategori   : Pekerjaan Profesional & Layanan Jasa
Lokasi     : Jalan Raya
Kesulitan  : 4
Reward     : 900
Status     : Available
```

Gambar 4.7 Output Tampilan Menyeluruh Daftar Informasi Isi Papan Mading Petualangan



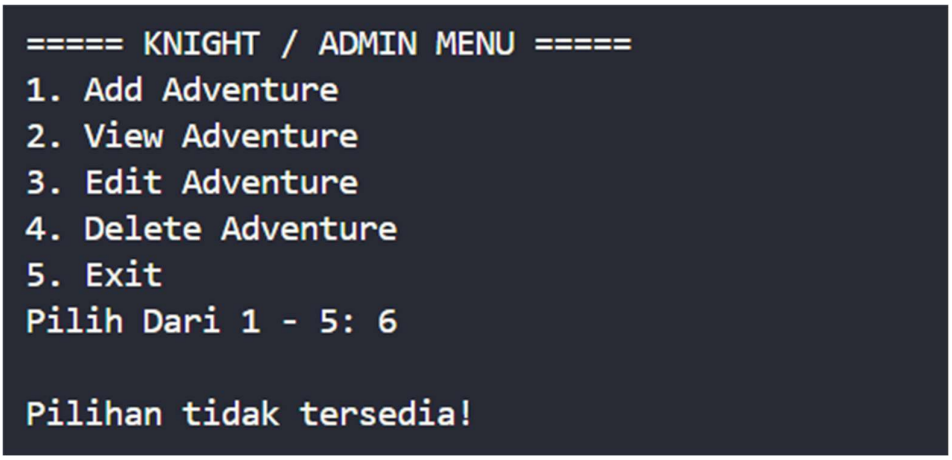
Gambar 4.8 Output Keberhasilan Login dan Tampilan Dasbor Untuk Peran Admin



Gambar 4.9 Output Pengisian Formulir dan Keberhasilan Penambahan Data Misi Petualangan Baru Serta Penolakan Sistem Jika Input Nilai Numerik Berupa Karakter/Huruf



Gambar 4.10 Output Notifikasi Kesalahan Jika Parameter Skala Kesulitan Melebihi Batas Aturan (1-5)



Gambar 4.11 Output Peringatan Sistem Jika Pilihan Angka Menu Dasbor Tidak Valid

```
===== KNIGHT / ADMIN MENU =====
1. Add Adventure
2. View Adventure
3. Edit Adventure
4. Delete Adventure
5. Exit
Pilih Dari 1 - 5: 3

========== UPDATE ADVENTURE ==========
Masukkan ID Adventure: A007
Adventure ditemukan!
Kosongkan nama atau lokasi jika tidak ingin mengubahnya.
Nama Baru:
Ubah kategori? (y/n)
Pilihan: n
Lokasi Baru:
Ubah kesulitan (y/n)
Pilihan: y
Tingkat Kesulitan (1-5): 2
Ubah reward (y/n)
Pilihan: y
Reward Baru: 85
Ubah status? (y/n)
Pilihan: n
Adventure berhasil diperbarui!

===== KNIGHT / ADMIN MENU =====
1. Add Adventure
2. View Adventure
3. Edit Adventure
4. Delete Adventure
5. Exit
Pilih Dari 1 - 5:
```

Gambar 4.12 Output Proses Pencarian ID dan Keberhasilan Penyuntingan (*Update*) Parameter Misi

```
ID          : A007
Pekerjaan   : Pemungut Sampah/Garbage Collector
Kategori    : Pekerjaan Sampingan & Paruh Waktu
Lokasi      : Hutan
Kesulitan   : 2
Reward      : 85
Status      : Available
```

Gambar 4.13 Hasil Dari *Update* Parameter Misi

```
========== DELETE ADVENTURE ==========
Masukkan ID Adventure: A007
Yakin Ingin Menghapus Pekerjaan Ini? (y/n): n
Penghapusan dibatalkan.

===== KNIGHT / ADMIN MENU =====
1. Add Adventure
2. View Adventure
3. Edit Adventure
4. Delete Adventure
5. Exit
Pilih Dari 1 - 5:
```

Gambar 4.14 Output Gerbang Konfirmasi Akhir dan Pembatalan Penghapusan (*Delete*) Data Misi

```
========== DELETE ADVENTURE ==========
Masukkan ID Adventure: A007
Yakin Ingin Menghapus Pekerjaan Ini? (y/n): y
Adventure berhasil dihapus!

===== KNIGHT / ADMIN MENU =====
1. Add Adventure
2. View Adventure
3. Edit Adventure
4. Delete Adventure
5. Exit
Pilih Dari 1 - 5:
```

Gambar 4.15 Output Gerbang Konfirmasi Akhir dan Keberhasilan Penghapusan (*Delete*) Data Misi

```
===== KNIGHT / ADMIN MENU =====
1. Add Adventure
2. View Adventure
3. Edit Adventure
4. Delete Adventure
5. Exit
Pilih Dari 1 - 5: 2

========== ADVENTURE BOARD ==========
1. Semua Pekerjaan
2. Filter Berdasarkan Kategori
3. Lihat Total Reward
Pilih 1 - 3: 3
Total reward seluruh adventure: 2980

===== KNIGHT / ADMIN MENU =====
1. Add Adventure
2. View Adventure
3. Edit Adventure
4. Delete Adventure
5. Exit
Pilih Dari 1 - 5:
```

Gambar 4.16 Output Hasil Kalkulasi Rekursif pada Menu Total Reward Seluruh Adventure

```
====================================
      CHIIKAWA ADVENTURE BOARD
====================================
1. Login
2. Register
3. Exit
Pilih Dari 1 - 3:
Program dihentikan.
PS C:\Users\honey>
```

Gambar 4.17 Output Pengecualian Global (Error Handling) Jika Terjadi Interupsi Penutupan Paksa (Ctrl+C)

```
195   def update_adventure():
237           print("2. Unavailable")
238           print("3. Taken")
239           status = input("Pilih Dari 1 - 3: ")
240
241           if status == "1":
242               job["status"] = "Available"
243           elif status == "2":
244               print(job["staaaaaatus"])
245           elif status == "3":
246               job["status"] = "Taken"
247           else:
248               print("Status tidak valid; status lama dipertahankan.")
249
250       print("Adventure berhasil diperbarui!")
251       return
```

Gambar 4.18 Potongan Kode Eksperimen pada Fungsi update_adventure() untuk Memicu Pengecualian KeyError

```
Ubah status? (y/n)
Pilihan: y
1. Available
2. Unavailable
3. Taken
Pilih Dari 1 - 3: 2
Terjadi kesalahan saat mengolah data: 'staaaaaatus'

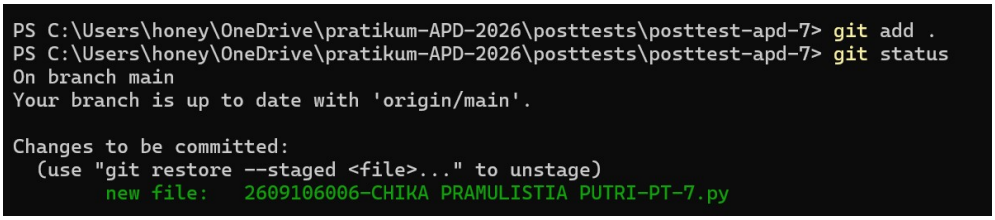
===== KNIGHT / ADMIN MENU =====
1. Add Adventure
2. View Adventure
3. Edit Adventure
4. Delete Adventure
5. Exit
Pilih Dari 1 - 5:
```

Gambar 4.19 Output Sistem Penanganan Kesalahan (Error Handling) Menu Admin saat Menangkap KeyError

5. Langkah-langkah Git

5. 1 Git Add

Fitur *git add* digunakan untuk memindahkan berkas kode program dari direktori kerja (*Working Directory*) menuju area penyiapan (*Staging Area*).

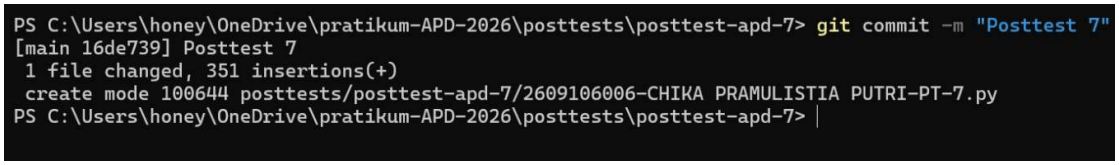


Gambar 5.1 Hasil Pemindahan Berkas ke *Staging Area* Menggunakan *git add*

Proses diawali dengan membuka terminal sistem di dalam folder repositori lokal pengerjaan tugas. Pengguna langsung mengeksekusi perintah *git add .* untuk memasukkan berkas program baru ke dalam antrean pelacakan Git secara massal. Setelah itu, perintah *git status* dijalankan untuk memastikan berkas telah berhasil dipindahkan. Proses ini dinyatakan sukses ketika terminal menampilkan nama berkas dengan teks berwarna hijau (*Changes to be committed*), menandakan berkas telah aman berada di *Staging Area* dan siap dikunci ke tahap selanjutnya.

5.2 Git Commit

Fitur *git commit* digunakan untuk menyimpan dan mengunci seluruh riwayat perubahan berkas dari *Staging Area* secara permanen ke dalam basis data repositori lokal.

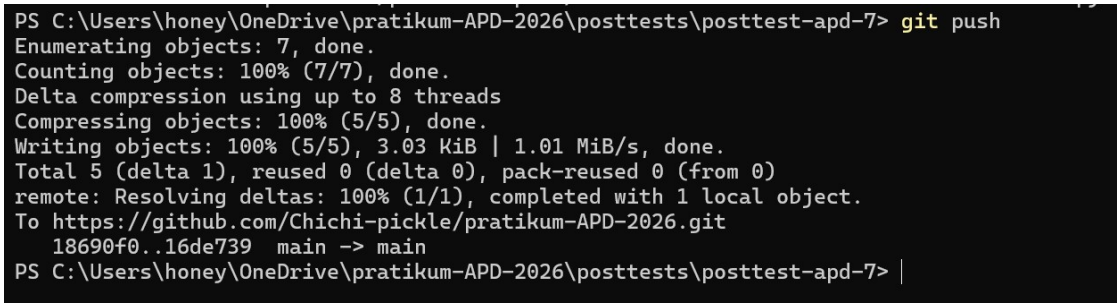


Gambar 5.2 Hasil Penguncian Riwayat Perubahan Melalui *git commit*

Setelah berkas dipastikan berwarna hijau pada pengecekan status, perintah *git commit -m "Posttest 5"* dijalankan. Flag *-m* berfungsi sebagai label dokumentasi untuk mencatat jenis perubahan yang dilakukan. Setelah menekan *Enter*, Git akan mengunci riwayat perubahan tersebut dan menampilkan ringkasan jumlah baris kode yang berhasil disimpan.

5.3 Git Push

Fitur git push digunakan untuk mengunggah dan menyelaraskan seluruh catatan riwayat commit dari repositori lokal ke dalam repositori daring di GitHub.



Gambar 5.3 Hasil Pengunggahan Berkas ke *GitHub* Melalui *git push*

Langkah terakhir ini berfungsi untuk mencadangkan berkas kode program secara daring ke platform *GitHub*. Pengunggahan dilakukan dengan mengeksekusi perintah *git push*. Terminal akan memproses pengiriman data ke peladen awan. Proses sinkronisasi ini dinyatakan berhasil sepenuhnya setelah terminal menampilkan log transfer data mencapai status 100% (*done*).